

WESTERN SYDNEY
UNIVERSITY



Module 7

Function Handles and Global Variables

Local and Global variables

- Variables that are created inside of a user-defined function may only be accessed from inside of that function. They are referred to as **local variables**.
- After the function completes its operations, the local variables **are deleted** from memory.
- The only variable that appears in the workspace is the output, if any, of the function.
- Conversely, functions **cannot** access variables from the workspace unless those are defined as **global variables**.

Local variables and local scope

- Variables inside and outside functions can have same names but are still different variables. Changing one doesn't change the other

```
function []=fun_ex1(x)
fprintf('At function start x = %f\n',x);
x = 100;
fprintf('At function end x = %f\n',x);
end
```

Function arguments are passed
by **value** not by **reference**!

```
>> x=1
x =
    1

>> fun_ex1(x)
At function start x = 1.000000
At function end x = 100.000000
>> x
x =
    1

>> fun_ex1(0.5)
At function start x = 0.500000
At function end x = 100.000000
>> x
x =
    1
```

Global Variables

- A variable defined as a **global** variable in the workspace have visibility to all functions. To define 'gravity' as a global variable
global gravity
- It is necessary to declare a variable to be **global** in every function that uses it
- **global** command must appear before the use of the variable

Example

```
function [y] = fun_ex2(vo,t)
global gravity
y = vo*t - 1/2*gravity*t^2;
end
```

```
>> global gravity
>> gravity=9.8
gravity =
    9.8000
>> y = fun_ex2(10,1.02)
y =
    5.1020
```

On use of global variables

- The use of global variables is **NOT** recommended – an incorrect change at one place will reflect in all places where it is used!
- It is also difficult to keep track of a large number of global variables
- There is always a way to write scripts so you do not need to use global variables
- There are some infrequent cases where global variables are good, for example, for the use of the commands **tic** and **toc**

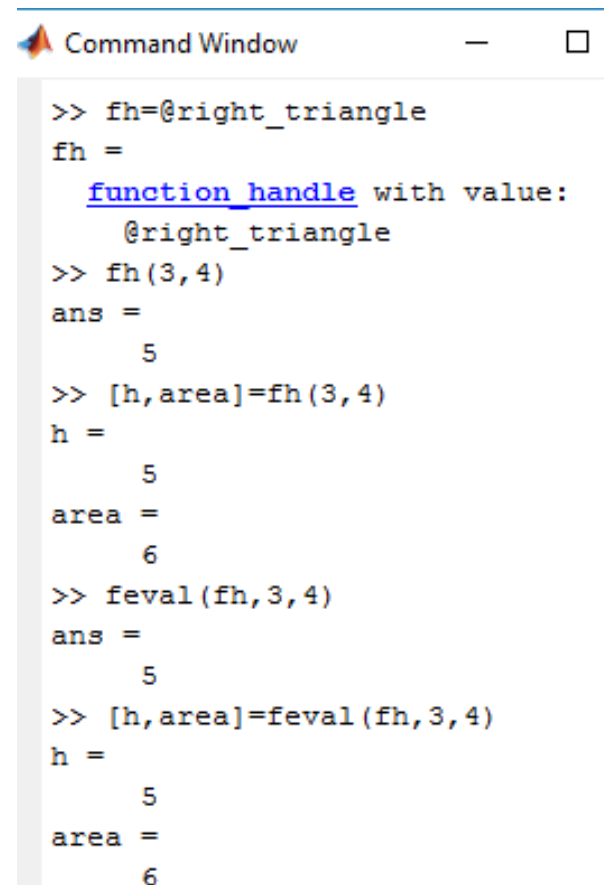
Function Handles

- A function handle is a MATLAB data type that stores an association to a function.
- Typical uses of function handles include:
 - Pass a function to another function
 - Specify callback functions in GUIs
 - Construct handles to functions defined inline instead of stored in a program file (anonymous functions)
 - Call local functions from outside the main function

Function Handles

- To create a handle for a function, precede the function name with an @ sign

```
>> fhandle=@sqrt
fhandle =
    function_handle with value:
        @sqrt
>> feval(fhandle,2)
ans =
    1.4142
>> fhandle(3)
ans =
    1.7321
```

A screenshot of the MATLAB Command Window. The window title is "Command Window". The command history shows: 1. Creating a function handle for 'right_triangle' and assigning it to 'fh'. 2. Calling 'fh(3,4)' which returns 5. 3. Calling '[h,area]=fh(3,4)' which returns h=5 and area=6. 4. Calling 'feval(fh,3,4)' which returns 5. 5. Calling '[h,area]=feval(fh,3,4)' which returns h=5 and area=6. The output for the first call is highlighted in blue.

```
Command Window

>> fh=@right_triangle
fh =
    function_handle with value:
        @right_triangle
>> fh(3,4)
ans =
    5
>> [h,area]=fh(3,4)
h =
    5
area =
    6
>> feval(fh,3,4)
ans =
    5
>> [h,area]=feval(fh,3,4)
h =
    5
area =
    6
```


Anonymous Functions

- An anonymous function is a **one-line expression-based MATLAB function** that does not require a program file
- Anonymous functions are only available while they are stored in the workspace window – much like variables
- The syntax

`h = @(arglist) anonymous_function`

Anonymous Functions - Examples

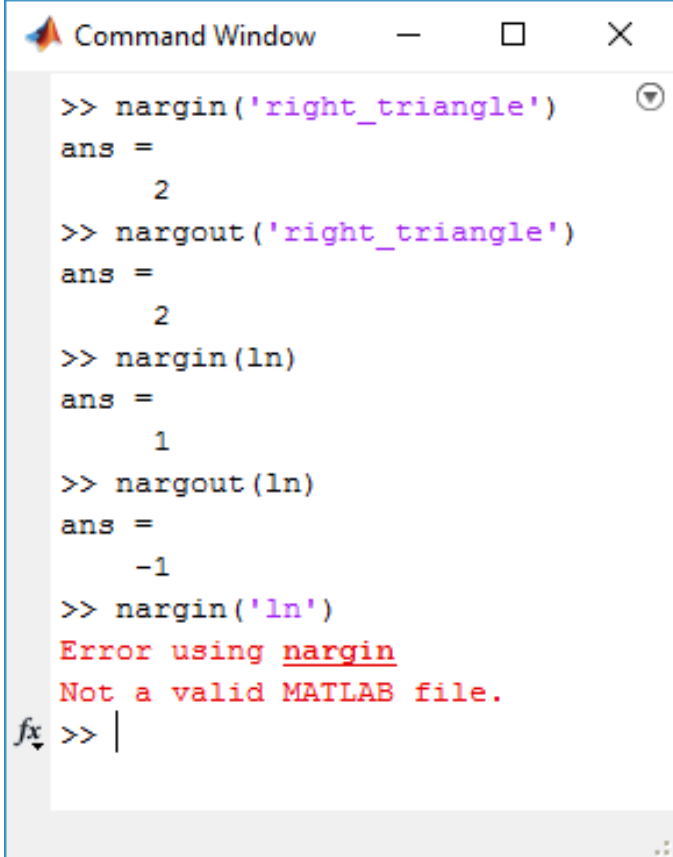
```
Command Window
>> sqr=@(x) x.^2
sqr =
    function_handle with value:
        @(x)x.^2
>> sqr(2)
ans =
     4
fx >>
```

```
Command Window
>> ln = @(x) log(x)
ln =
    function_handle with value:
        @(x)log(x)
>> ln(2)
ans =
    0.6931
fx >> |
```

```
Command Window
>> rt=@(a,b) [sqrt(a^2+b^2), 1/2*a*b]
rt =
    function_handle with value:
        @(a,b)[sqrt(a^2+b^2),1/2*a*b]
>> rt(3,4)
ans =
     5     6
fx >> |
```

Determining the number of input and output arguments

- `nargin`
 - determines the number of input arguments
- `nargout`
 - determines the number of output arguments
- The input to these functions is represented using strings unless those are function handles

A screenshot of the MATLAB Command Window. The window title is "Command Window". It shows a series of commands and their outputs. The first two commands, `nargin('right_triangle')` and `nargout('right_triangle')`, both return the value 2. The next two commands, `nargin(ln)` and `nargout(ln)`, both return the value 1. The final command is `nargin('ln')`, which results in an error message: "Error using nargin Not a valid MATLAB file." The cursor is at the prompt `>>` on the line following the error.

```
>> nargin('right_triangle')
ans =
     2
>> nargout('right_triangle')
ans =
     2
>> nargin(ln)
ans =
     1
>> nargout(ln)
ans =
    -1
>> nargin('ln')
Error using nargin
Not a valid MATLAB file.
fx >> |
```

Function definitions in a MATLAB script file – local functions

- MATLAB scripts can contain function definitions. This is a new feature since MATLAB 2016B but was not mainstream until recently.
- These functions are local to the script file, i.e., they cannot be accessed from outside.
- Before 2024A version, local function should be defined at the end of the file, after the last line of the script.

```
x = 1:10;  
funcSum = myCal(x)  
  
function y = myCal(x)  
y = sum(x.^2 + 2*x + 7);  
end
```